REST API Development

A practical guide to designing, building, securing and documenting RESTful APIs

Covers: REST principles • HTTP methods & status codes • resource design • building an API with Node.js/Express • validation • authentication • versioning • pagination • testing • documentation • best practices

Contents

1. Introduction to REST
2. HTTP Methods and Status Codes
3. Designing Resources and URLs
4. Request and Response Format
5. Building a REST API with Express
6. Validation and Error Handling
7. Authentication and Authorization
8. Pagination, Filtering and Sorting
9. Versioning
10. Testing REST APIs
11. Documentation with OpenAPI
12. Security and Performance Best Practices
13. Quick Reference Checklist

1. Introduction to REST

REST (Representational State Transfer) is an architectural style for networked applications, introduced by Roy Fielding in 2000. A REST API exposes *resources* (users, orders, products) over HTTP, and clients interact with them using standard HTTP methods.

The six REST constraints

Constraint	Meaning
Client-Server	UI and data storage concerns are separated; each can evolve independently.
Stateless	Each request contains everything needed to process it. The server stores no client session state.
Cacheable	Responses declare whether they can be cached, improving performance and scalability.
Uniform Interface	Resources are identified by URIs and manipulated through standard methods and representations.
Layered System	Clients cannot tell whether they talk to the origin server or an intermediary (proxy, gateway, CDN).
Code on Demand (optional)	Servers may send executable code (e.g. JavaScript) to the client.

REST vs other API styles

Style	Strengths	Typical use
REST	Simple, cacheable, widely supported	Public APIs, CRUD services, mobile backends
GraphQL	Client picks fields; single endpoint	Complex, nested data needs
gRPC	Binary, fast, strongly typed	Internal microservice communication
SOAP	Strict contracts, WS-* standards	Legacy enterprise integrations

2. HTTP Methods and Status Codes

Method	Purpose	Safe	Idempotent
GET	Retrieve a resource or collection	Yes	Yes
POST	Create a resource / trigger an action	No	No
PUT	Replace a resource entirely	No	Yes
PATCH	Partially update a resource	No	No (usually)
DELETE	Remove a resource	No	Yes
HEAD / OPTIONS	Metadata / supported methods (CORS)	Yes	Yes

Safe means the request does not change server state. **Idempotent** means repeating the request has the same effect as sending it once.

Common status codes

Code	Name	When to use
200	OK	Successful GET, PUT, PATCH
201	Created	Resource created; include a Location header
204	No Content	Successful DELETE or update with no body
400	Bad Request	Malformed syntax or invalid input
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated but not permitted
404	Not Found	Resource does not exist
409	Conflict	State conflict, e.g. duplicate email
422	Unprocessable Entity	Well-formed but semantically invalid data
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected server failure
503	Service Unavailable	Temporarily overloaded or in maintenance

3. Designing Resources and URLs

- Use **nouns**, not verbs: `/users`, not `/getUsers`.
- Use **plural** names for collections: `/products`, `/products/42`.
- Express relationships through nesting (max 2 levels): `/users/7/orders`.
- Use lowercase and hyphens: `/order-items`.
- Keep query strings for filtering, sorting and pagination.
- Let the HTTP method express the action.

```
GET    /api/v1/users            -> list users
POST   /api/v1/users            -> create a user
GET    /api/v1/users/42         -> get user 42
PUT    /api/v1/users/42         -> replace user 42
PATCH  /api/v1/users/42         -> update fields of user 42
DELETE /api/v1/users/42         -> delete user 42
GET    /api/v1/users/42/orders  -> orders belonging to user 42
```

4. Request and Response Format

JSON is the de facto standard. Send `Content-Type: application/json` with requests and use consistent field naming (camelCase or snake_case, but not both).

```
POST /api/v1/users HTTP/1.1
Content-Type: application/json

{ "name": "Asha Rao", "email": "asha@example.com" }

HTTP/1.1 201 Created
Location: /api/v1/users/101
Content-Type: application/json

{ "id": 101, "name": "Asha Rao", "email": "asha@example.com",
  "createdAt": "2026-10-28T09:30:00Z" }
```

Consistent error format

```
HTTP/1.1 422 Unprocessable Entity

{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "One or more fields are invalid.",
    "details": [ { "field": "email", "issue": "must be a valid email" } ]
  }
}
```

5. Building a REST API with Express

This example builds a small in-memory users API with Node.js and Express.

```
mkdir users-api && cd users-api
npm init -y
npm install express

// server.js
const express = require("express");
const app = express();
app.use(express.json());

let users = [{ id: 1, name: "Asha", email: "asha@example.com" }];
let nextId = 2;

// List
app.get("/api/v1/users", (req, res) => res.json(users));

// Read one
app.get("/api/v1/users/:id", (req, res) => {
  const user = users.find(u => u.id === Number(req.params.id));
  if (!user) return res.status(404).json({ error: { message: "User not found" } });
  res.json(user);
});

// Create
app.post("/api/v1/users", (req, res) => {
  const { name, email } = req.body;
  const user = { id: nextId++, name, email };
  users.push(user);
  res.status(201).location(`/api/v1/users/${user.id}`).json(user);
});

// Update (partial)
app.patch("/api/v1/users/:id", (req, res) => {
  const user = users.find(u => u.id === Number(req.params.id));
  if (!user) return res.status(404).json({ error: { message: "User not found" } });
  Object.assign(user, req.body);
  res.json(user);
});

// Delete
app.delete("/api/v1/users/:id", (req, res) => {
  users = users.filter(u => u.id !== Number(req.params.id));
  res.status(204).end();
});

app.listen(3000, () => console.log("API running on http://localhost:3000"));
```

Run with `node server.js` and try it:

```
curl -X POST http://localhost:3000/api/v1/users \
  -H "Content-Type: application/json" \
  -d '{"name":"Ravi","email":"ravi@example.com"}'
```

Recommended project structure

```
src/
  routes/        users.routes.js
  controllers/   users.controller.js
  services/      users.service.js      (business logic)
  models/        user.model.js         (DB access)
  middleware/    auth.js, validate.js, errorHandler.js
  config/        env.js, db.js
  app.js         server.js
tests/
```

6. Validation and Error Handling

Never trust client input. Validate every request body, query parameter and path parameter. Libraries such as **Joi**, **Zod** or **express-validator** make this straightforward.

```
const { z } = require("zod");

const createUserSchema = z.object({
  name: z.string().min(2),
  email: z.string().email()
});

function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body);
    if (!result.success) {
      return res.status(422).json({
        error: { code: "VALIDATION_FAILED", details: result.error.issues }
      });
    }
    req.body = result.data;
    next();
  };
}

app.post("/api/v1/users", validate(createUserSchema), createUser);
```

Centralised error handler

```
// must be registered AFTER all routes
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({
    error: { code: err.code || "INTERNAL_ERROR",
             message: err.status ? err.message : "Something went wrong" }
  });
});
```

Do not leak stack traces or database errors to clients in production.

7. Authentication and Authorization

Method	Description	Best for
API Keys	Static key sent in a header	Server-to-server, simple integrations
Basic Auth	Base64 username:password (HTTPS only)	Internal tools, prototypes
JWT (Bearer)	Signed token carrying user claims	Stateless user authentication
OAuth 2.0 / OIDC	Delegated authorisation via tokens	Third-party access, social login

JWT example

```
const jwt = require("jsonwebtoken");
```

```
// Login: issue a token
app.post("/api/v1/auth/login", async (req, res) => {
  const user = await verifyCredentials(req.body);   // check hashed password
  if (!user) return res.status(401).json({ error: { message: "Invalid credentials" } });
  const token = jwt.sign({ sub: user.id, role: user.role },
                         process.env.JWT_SECRET, { expiresIn: "15m" });
  res.json({ accessToken: token });
});

// Middleware: protect routes
function authenticate(req, res, next) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: { message: "Token required" } });
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ error: { message: "Invalid or expired token" } }); }
}

function authorize(...roles) {
  return (req, res, next) =>
    roles.includes(req.user.role) ? next()
      : res.status(403).json({ error: { message: "Forbidden" } });
}

app.delete("/api/v1/users/:id", authenticate, authorize("admin"), deleteUser);
```

- Always hash passwords with **bcrypt** or **argon2**; never store plain text.
- Use short-lived access tokens plus refresh tokens.
- Serve everything over HTTPS.
- **401** = not authenticated; **403** = authenticated but not allowed.

8. Pagination, Filtering and Sorting

```
GET /api/v1/products?page=2&limit=20
GET /api/v1/products?category=books&minPrice=100&sort=-price
GET /api/v1/products?fields=id,name,price

{
  "data": [ { "id": 21, "name": "..." } ],
  "meta": { "page": 2, "limit": 20, "total": 134, "totalPages": 7 },
  "links": { "next": "/api/v1/products?page=3&limit=20",
             "prev": "/api/v1/products?page=1&limit=20" }
}
```

Strategy	Pros	Cons
Offset (page/limit)	Simple; jump to any page	Slow on large tables; inconsistent when data changes
Cursor-based	Fast and stable for large or live data	No random page access; more complex

9. Versioning

Approach	Example
URI path (most common)	/api/v1/users
Header	Accept: application/vnd.myapp.v2+json
Query parameter	/api/users?version=2

- Only bump the major version for **breaking** changes (removing or renaming fields, changing types).
- Adding optional fields or new endpoints is non-breaking.
- Announce deprecations early and use the `Deprecation` and `Sunset` headers.

10. Testing REST APIs

- **Unit tests** for services and utilities.
- **Integration tests** that call endpoints against a test database.
- **Contract tests** to verify the API matches its OpenAPI spec.
- **Load tests** (k6, JMeter) to find performance limits.
- Manual exploration with Postman, Insomnia, Bruno or curl.

```
// users.test.js  (Jest + Supertest)
const request = require("supertest");
const app = require("../src/app");

describe("POST /api/v1/users", () => {
  it("creates a user", async () => {
    const res = await request(app)
      .post("/api/v1/users")
      .send({ name: "Meera", email: "meera@example.com" });
    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty("id");
  });

  it("rejects an invalid email", async () => {
    const res = await request(app).post("/api/v1/users")
      .send({ name: "Meera", email: "nope" });
    expect(res.status).toBe(422);
  });
});
```

11. Documentation with OpenAPI

OpenAPI (formerly Swagger) is the standard, machine-readable format for describing REST APIs. From one spec you can generate interactive docs (Swagger UI, Redoc), client SDKs and mock servers.

```
openapi: 3.0.3
info:
  title: Users API
  version: 1.0.0
paths:
  /users/{id}:
    get:
      summary: Get a user by ID
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer }
      responses:
        "200":
          description: The user
          content:
            application/json:
              schema: { $ref: "#/components/schemas/User" }
        "404":
          description: User not found
components:
  schemas:
```

```
    User:
      type: object
      properties:
        id:    { type: integer }
        name:  { type: string }
        email: { type: string, format: email }
```

12. Security and Performance Best Practices

Security

- Enforce HTTPS and set security headers (use **helmet** in Express).
- Apply rate limiting (**express-rate-limit**) to prevent abuse and brute force.
- Configure CORS to allow only trusted origins.
- Use parameterised queries or an ORM to prevent SQL injection.
- Limit request body size and validate content types.
- Keep secrets in environment variables or a secrets manager, never in source control.
- Follow the OWASP API Security Top 10, especially broken object-level authorisation (check that users own the resource they request).
- Log requests, but never log passwords, tokens or personal data.

Performance

- Enable gzip/brotli compression.
- Use HTTP caching: `Cache-Control`, `ETag`, conditional requests (304 Not Modified).
- Cache hot data with Redis; add database indexes for common queries.
- Always paginate list endpoints; avoid N+1 queries.
- Use async processing (queues) for slow work and return **202 Accepted**.

```
const helmet = require("helmet");
const rateLimit = require("express-rate-limit");
const cors = require("cors");

app.use(helmet());
app.use(cors({ origin: ["https://myapp.example.com"] }));
app.use(rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
```

13. Quick Reference Checklist

- [ ] Resources are plural nouns; actions are expressed by HTTP methods
- [ ] Correct status codes returned (201 on create, 204 on delete, 4xx for client errors)
- [ ] Consistent JSON structure and error format
- [ ] Input validated on every endpoint
- [ ] Authentication and role-based authorisation in place
- [ ] HTTPS, CORS, rate limiting and security headers configured
- [ ] List endpoints paginated, filterable and sortable
- [ ] API versioned (e.g. /v1) with a deprecation policy

- [ ] Automated tests cover happy paths and error cases
- [ ] OpenAPI documentation published and kept up to date
- [ ] Logging, monitoring and health-check endpoint (/health)

Suggested next steps

- Connect the API to a real database (PostgreSQL/MongoDB) using an ORM such as Prisma or Sequelize.
- Containerise with Docker and set up CI/CD.
- Explore HATEOAS, webhooks and idempotency keys for advanced designs.